BÖLÜM 3 / MODULE 3

Vektörler, Embedding'ler ve Vektör Veritabanları

Vectors, Embeddings & Vector Database Architecture

Embedding modelleri, metinlerin yüksek boyutlu uzayda haritalanması, Cosine Similarity ve Dot Product metrikleri, HNSW graf indeksleme, pgvector ve kurumsal vektör veritabanı seçimi.

Embedding models, dense high-dimensional semantic spaces, Cosine Similarity vs Dot Product, HNSW graph indexing algorithms, pgvector, and enterprise vector database trade-offs.

13

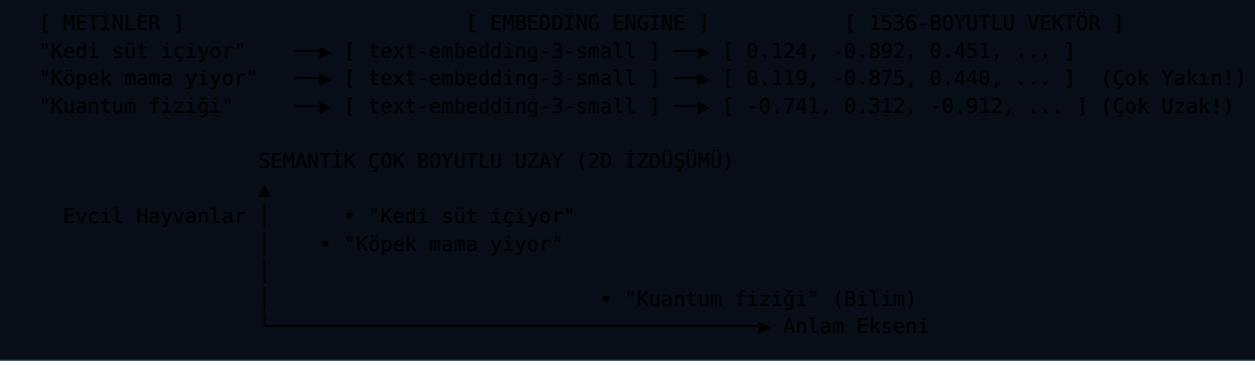
BÖLÜM: VEKTÖRLER, EMBEDDING'LER VE VEKTÖR VERITABANLARI | MODULE: VECTORS, EMBEDDINGS & VECTOR DATABASE ARCHITECTURE

Embedding Nedir? Metinlerin Çok Boyutlu Matematiksel Haritası

Demystifying Embeddings: Mapping Semantic Meaning into Dense Vector Space

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanedeki kitapları sadece adına göre sıralarsanız, 'Kedi Bakımı' ile 'Köpek Eğitimi' kitapları farklı raflara düşer. Ama kitapları konusuna göre 3 boyutlu bir haritada yerleştirirseniz: Birinci eksen 'Hayvan Sevgisi', ikinci eksen 'Yemek', üçüncü eksen 'Bilim' olsun. Kedi ve köpek kitapları haritada yan yana dururken, 'Kuantum Fiziği' odanın öbür ucuna uçar! İşte Embedding tam olarak budur: Bilgisayarın kelimelerin anlamını anlayabilmesi için her cümleye 1536 adet koordinat sayısı (Vektör) vermesidir!

🇬🇧 ENGLISH (ELI5)

Alphabetical library sorting places 'Cat Nutrition' and 'Dog Training' far apart under C and D. Now imagine placing books in a 3D coordinate room where Axis-X is 'Pet Care', Axis-Y is 'Dietary', and Axis-Z is 'Astrophysics'. Pet books cluster closely together, while 'Quantum Mechanics' sits on the opposite wall! That is an Embedding: a neural transformation converting unstructured human prose into a dense array of floating-point coordinates (1,536 dimensions) where semantic proximity equals geometric proximity!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Embedding Modellerinin Teknik Detayları: 1) Boyutluluk (Dimensionality): OpenAI `text-embedding-3-small` 1536 boyutlu, `text-embedding-3-large` ise 3072 boyutlu vektörler üretir. Boyut arttıkça anlam hassasiyeti artar ancak depolama ve RAM maliyeti katlanır. 2) Matryoshka Embeddings: OpenAI'ın v3 modelleri, Rus matruşka bebekleri gibi ilk 512 boyutu alıp geri kalanını kırpmanıza (Truncation) izin verir; doğruluk %98 korunurken veritabanı boyutu %66 küçülür! 3) Normalizasyon: OpenAI vektörleri birim uzunluktadır (||v|| = 1); bu sayede pahalı Cosine Similarity yerine süper hızlı Dot Product (İç Çarpım) kullanılabilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Embedding Mathematics and Economics: 1) Dense Vector Topology: `text-embedding-3-small` outputs 1,536-dimensional float32 arrays (6KB per vector). `text-embedding-3-large` scales to 3,072 dimensions for subtle multi-domain nuances. 2) Matryoshka Representation Learning (MRL): Modern v3 embeddings allow truncation to lower dimensions (e.g. 512 or 256) while retaining >95% retrieval accuracy, slashing RAM and storage footprint by 66%. 3) Unit Normalization: Vectors are pre-normalized to unit magnitude (L2 norm = 1.0). Consequently, Cosine Distance simplifies mathematically to a raw Dot Product (u · v), reducing CPU vector search cycles.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: Metinleri Vektör Embedding'e Dönüştürme
import org.springframework.ai.embedding.EmbeddingModel;
import java.util.List;

public class EmbeddingGeneratorService {
    private final EmbeddingModel embeddingModel;

    public EmbeddingGeneratorService(EmbeddingModel embeddingModel) {
        this.embeddingModel = embeddingModel;
    }

    public float[] generateVector(String text) {
        // OpenAI text-embedding-3-small çağrısı yapar ve 1536 float döner
        return embeddingModel.embed(text);
    }

    public List<float[]> batchEmbed(List<String> documents) {
        return embeddingModel.embed(documents);
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Ölümcül Hata: Dokümanları `text-embedding-3-small` (1536 boyut) ile indeksleyip, arama yaparken sorguyu `text-embedding-3-large` (3072 boyut) veya Cohere ile vektörleştiremezsiniz! İndeksleme ve sorgulama modelleri BİREBİR AYNI model ve boyut olmak zorundadır.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Catastrophic error: Searching an index embedded with `text-embedding-3-small` using queries embedded via `text-embedding-3-large` or HuggingFace. Vector dimensions and semantic manifolds must match 100% identically across ingestion and query pipelines.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Embedding, metinlerin anlamsal özünü çok boyutlu koordinat sayılarına dönüştürür.

• Benzer anlama gelen cümleler vektör uzayında geometrik olarak birbirine çok yakındır.

• Matryoshka boyutsallık kırpma, doğruluktan ödün vermeden RAM maliyetini %66 düşürür.

🎯 Key Takeaways for the Interview

• Embeddings compress semantic human intent into dense high-dimensional vectors.

• Semantically related sentences cluster tightly in vector space.

• Matryoshka representation learning allows truncation to slash vector memory spend by 66%.

14

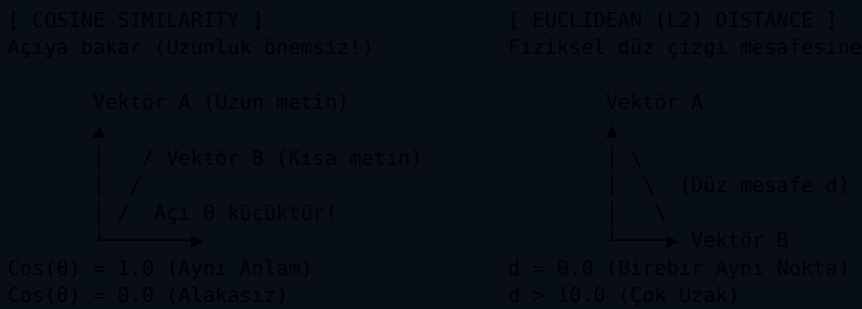
BÖLÜM: VEKTÖRLER, EMBEDDING'LER VE VEKTÖR VERITABANLARI | MODULE: VECTORS, EMBEDDINGS & VECTOR DATABASE ARCHITECTURE

Vektör Benzerlik Metrikleri: Cosine Similarity, Dot Product & Euclidean Distance

Vector Distance Metrics: Cosine Similarity, Dot Product & Euclidean (L2) Distance

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İki ok düşünün: Biri 10 metre uzunluğunda devasa bir mızrak (uzun bir paragraf), diğeri 10 santimlik minik bir iğne (kısa bir başlık). İkisi de tam olarak aynı yöne (kuzeye) bakıyorsa, konuları BİREBİR AYNIDIR! Cosine Similarity boya veya uzunluğa bakmaz; sadece iki okun arasındaki açıya bakar! Açı sıfırsa benzerlik 1'dir. Euclidean Distance ise iki okun uçları arasındaki cetvel mesafesini ölçer. Arama motorlarında metin uzunlukları farklı olabildiği için her zaman Cosine Similarity tercih edilir!

🇬🇧 ENGLISH (ELI5)

Imagine two arrows pointing due North: one is a 10-foot wooden javelin (a comprehensive 500-word article), the other is a 2-inch dart (a 5-word headline). Because they point in the exact same direction, their semantic topic is identical! Cosine Similarity measures strictly the angle between vectors, completely ignoring document length magnitude. Euclidean (L2) distance measures absolute spatial distance with a ruler. For textual retrieval where document chunk sizes vary, Cosine Distance is the industry standard!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Matematiksel Formüller ve Donanım Optimizasyonu: 1) Cosine Similarity: Cos(θ) = (u · v) / (||u|| · ||v||). Sonuç −1 ile +1 arasındadır. 2) Dot Product: u · v = Σ(u_i · v_i). Eğer vektörler birim normalizasyona sahipse (||u|| = ||v|| = 1), Cosine Similarity ile Dot Product BİREBİR AYNI sonucu verir! Dot product karekök ve bölme işlemi gerektirmediği için CPU SIMD / AVX-512 ve GPU'larda 5 kat daha hızlı hesaplanır. 3) Euclidean (L2): d = √(Σ (u_i - v_i)²). Boyut büyüklüğüne duyarlıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Vector metric mechanics and hardware acceleration: 1) Cosine Similarity: Evaluates orientation (Cos(θ) = (u · v) / (||u|| · ||v||)), bounding values between −1.0 and 1.0. 2) Dot Product Optimization: If embedding vectors are normalized to unit length (L2 = 1.0), Cosine Similarity is algebraically identical to the raw Dot Product (u · v). Eliminating the magnitude denominator allows modern CPUs to leverage AVX-512 SIMD vector instructions, accelerating distance calculations by 500%. 3) Euclidean (L2): Measures point-to-point Pythagorean distance; sensitive to text magnitude.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25: İki Vektör Arasındaki Cosine Similarity ve Dot Product Hesaplama
public class VectorDistanceCalculator {
    public static double cosineSimilarity(float[] vectorA, float[] vectorB) {
        if (vectorA.length != vectorB.length) {
            throw new IllegalArgumentException("Vektör boyutları eşit olmalıdır!");
        }

        double dotProduct = 0.0;
        double normA = 0.0;
        double normB = 0.0;

        for (int i = 0; i < vectorA.length; i++) {
            dotProduct += vectorA[i] * vectorB[i];
            normA += vectorA[i] * vectorA[i];
            normB += vectorB[i] * vectorB[i];
        }

        if (normA == 0.0 || normB == 0.0) return 0.0;
        return dotProduct / (Math.sqrt(normA) * Math.sqrt(normB));
    }
}

void main() {
    float[] query = {0.8f, 0.6f, 0.0f};
    float[] docA  = {0.8f, 0.6f, 0.0f}; // Birebir aynı yön
    float[] docB  = {-0.6f, 0.8f, 0.0f}; // Dik açı (Alakasız)

    IO.println("Sorgu - DocA Benzerliği: " + VectorDistanceCalculator.cosineSimilarity(query,
docA)); // 1.0
    IO.println("Sorgu - DocB Benzerliği: " + VectorDistanceCalculator.cosineSimilarity(query,
docB)); // 0.0
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Vektör Veritabanı Tuzağı: pgvector veya Pinecone'da indeks oluştururken metriği yanlış seçmek (örneğin normalize edilmiş OpenAI vektörleri için Cosine yerine L2 Distance seçmek) hem arama kalitesini düşürür hem de sorguları yavaşlatır. OpenAI için daima `vector_cosine_ops` veya `vector_ip_ops` (Inner Product) seçilmelidir.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Database index pitfall: Selecting L2 distance in pgvector when indexing normalized OpenAI vectors degrades search throughput. Always specify the inner product index (`vector_ip_ops`) for maximum SIMD efficiency.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Cosine Similarity metin uzunluğundan bağımsız olarak anlamsal açıyı ölçer.
- Normalize edilmiş vektörlerde Dot Product, Cosine ile aynıdır ama çok daha hızlıdır.
- Vektör veritabanı indeksi kurulurken embedding modelinin beklediği metrik seçilmelidir.

🎯 Key Takeaways for the Interview

- Cosine Similarity measures semantic directionality independent of document length.
- For normalized vectors, Dot Product equals Cosine Similarity with dramatically lower CPU cost.
- Vector database index operators must explicitly match your embedding model's properties.

15

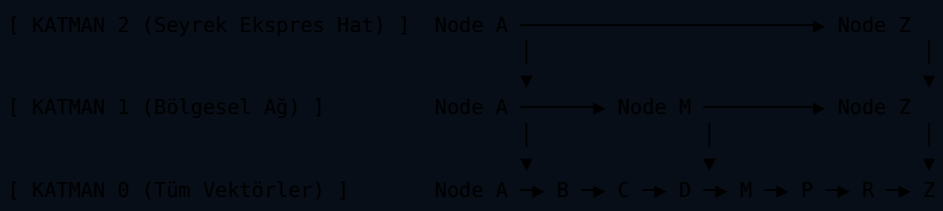
BÖLÜM: VEKTÖRLER, EMBEDDING'LER VE VEKTÖR VERITABANLARI | MODULE: VECTORS, EMBEDDINGS & VECTOR DATABASE ARCHITECTURE

Vektör İndeksleme Algoritmaları: HNSW (Hierarchical Navigable Small World) & IVF

Approximate Nearest Neighbors (ANN): Inside HNSW & IVF Indexing Algorithms

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

1 milyon kişilik bir şehirde Ali'nin evini aradığınızı düşünün. Tek tek 1 milyon evin kapısını çalarsanız (Kaba Kuvvet / Brute-Force O(N)) aylar sürer! Bunun yerine önce uçağa binip şehre kuş bakışı bakarsınız (Katman 2), sonra doğru semte inip arabayla ana caddeye girersiniz (Katman 1), en son sokaktaki kapı numarasına bakarsınız (Katman 0)! HNSW (Hierarchical Navigable Small World) algoritması tam olarak bu otoban sistemidir: Milyonlarca vektör arasında aradığınız en yakın 5 dokümanı 2 milisaniyede bulur!

🇬🇧 ENGLISH (ELI5)

Locating a specific house among 1,000,000 homes by knocking on every door sequentially takes months (Brute-Force Flat Search O(N)). Instead, you take an interstate highway across regions (Layer 2), exit onto an arterial boulevard into the right neighborhood (Layer 1), and walk the final residential street to the exact address (Layer 0). Hierarchical Navigable Small World (HNSW) organizes vector space into this multi-layer highway graph, executing Approximate Nearest Neighbor (ANN) searches across millions of embeddings in sub-5ms (O(log N))!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Vektör İndeks Algoritmalarının Karşılaştırması: 1) Flat Index (Kaba Kuvvet): %100 kesin sonuç verir (Recall 1.0) ancak her aramada tüm vektörlerle tek tek iç çarpım yapar (O(N)). 100k vektörden sonra çöker. 2) IVF (Inverted File Index): Vektörleri k-means ile Voronoi hücrelerine kümeler; sorgu sadece en yakın 3-5 kümede aranır. Hızlıdır ancak güncellemesi (insert) zordur. 3) HNSW (Skip-List Tabanlı Graf): Günümüzün altın standardıdır. Hem yüksek Recall (%98+) sunar hem de arama süresi O(log N)'dir. Tek dezavantajı: Graf bağlantılarını tuttuğu için RAM tüketimi %20-%30 daha fazladır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Vector Index Engineering & Trade-offs: 1) Flat Index (Exact Brute-Force): Guarantees 100% recall via exhaustive comparison, but scales at O(N · D). Unusable past 50,000 vectors at production SLAs. 2) IVF (Inverted File Index): Partitions vector space into Voronoi centroids using K-means clustering. Prunes search to top n clusters; sensitive to centroid drift under continuous updates. 3) HNSW (Hierarchical Navigable Small World): Multi-layer skip-graph architecture. Achieves >98% recall with O(log N) search latency. Primary trade-off: higher RAM consumption to retain graph edge pointers.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: pgvector için HNSW İndeks DDL SQL Tanımı
public class VectorIndexConfiguration {
    public static final String CREATE_HNSW_INDEX_SQL = """
        -- 1536 Boyutlu OpenAI Vektörleri için HNSW İndeksi Oluşturma
        CREATE INDEX IF NOT EXISTS document_embedding_hnsw_idx
        ON document_store
        USING hnsw (embedding vector_cosine_ops)
        WITH (
            m = 16,               -- Düğüm başına maksimum komşu kenar sayısı (Default: 16)
            ef_construction = 64  -- İndeks inşa edilirken taranacak aday listesi boyutu
        );
        """;

    public static final String QUERY_HNSW_SQL = """
        -- HNSW Arama Sırasında ef_search ayarı (Hız vs Recall dengesi)
        SET hnsw.ef_search = 40;

        SELECT id, content, 1 - (embedding <=> :queryVector) AS similarity
        FROM document_store
        ORDER BY embedding <=> :queryVector
        LIMIT 5;
        """;
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Üretim Uyarısı: HNSW indeksini 1 milyon kayıt yüklendikten sonra sıfırdan oluşturmak devasa CPU tüketir ve saatlerce sürebilir. Veri büyümeden önce indeksi baştan tanımlayın veya veriyi parça parça (Batch) yükleyin.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Operational hazard: Constructing an HNSW index on a pre-populated table with millions of vectors can saturate CPU and lock database memory for hours. Provision adequate build memory (`maintenance_work_mem`) and index incrementally.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Brute-Force arama üretimde ölçeklenmez; ANN (Yaklaşık En Yakın Komşu) zorunludur.

• HNSW, endüstride hız ve doğruluk (Recall) dengesi en iyi olan algoritmadır.

• m ve ef parametreleri RAM kullanımı ile arama kalitesi arasındaki takası belirler.

🎯 Key Takeaways for the Interview

• Exhaustive brute-force search is unviable at scale; ANN graph indexing is mandatory.

• HNSW provides the industry's premier trade-off between recall accuracy and low query latency.

• Tuning `m` and `ef` balances memory overhead against search precision.

16

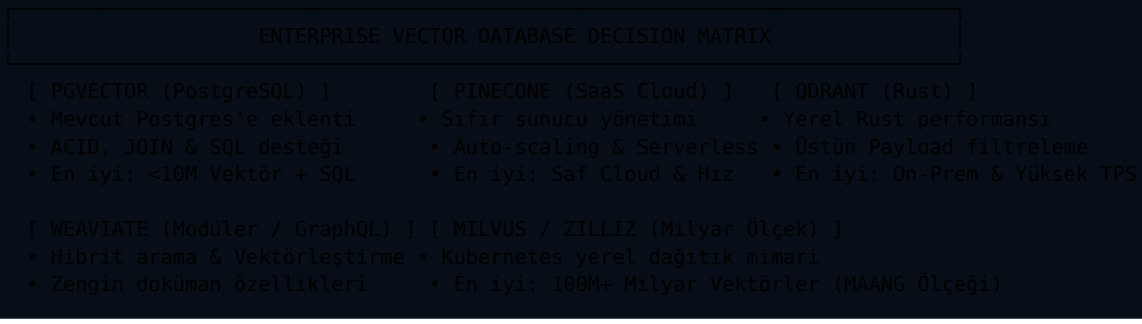
BÖLÜM: VEKTÖRLER, EMBEDDING'LER VE VEKTÖR VERITABANLARI | MODULE: VECTORS, EMBEDDINGS & VECTOR DATABASE ARCHITECTURE

Vektör Veritabanı Seçimi: pgvector vs Pinecone vs Qdrant vs Weaviate vs Milvus

Vector Database Evaluation: pgvector vs Pinecone vs Qdrant vs Weaviate vs Milvus

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Vektör veritabanı seçmek araba seçmek gibidir: Eğer zaten garajınızda sağlam bir aile arabanız varsa (PostgreSQL), arkasına bir çeki demiri takıp (pgvector eklentisi) 100.000 dokümanınızı rahatça taşıyabilirsiniz; ekstra masrafa gerek yoktur! Ama bir lojistik şirketiyseniz ve her gün 500 milyon koli taşıyacaksanız, özel bir tır filosu (Milvus veya Pinecone) kiralamanız gerekir. MAANG mülakatında 'Hemen Pinecone kuralım' derseniz elenirsiniz; 'Önce veri hacmimiz kaç? Mevcut ilişkisel verimizle JOIN yapacak mıyız?' diye sormanız gerekir!

🇬🇧 ENGLISH (ELI5)

Selecting a vector store is like choosing transportation: If you already run a reliable enterprise fleet of delivery vans (PostgreSQL), installing a trailer hitch (the `pgvector` extension) handles 500,000 embeddings with ACID compliance and zero architectural overhead. But if you are managing a global airline logistics network with 500 million packages, you deploy purpose-built cargo jets (Pinecone or Milvus). In MAANG interviews, never blindly default to specialized vector SaaS without evaluating operational complexity!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Karar Matrisi Kriterleri: 1) pgvector: Eğer doküman sayısı 5 milyonun altındaysa ve kullanıcı tablosu, faturalar veya yetkilendirme (RBAC) ile SQL JOIN gerekiyorsa pgvector tartışmasız en iyi seçimdir. Ayrı bir veritabanı yönetme ve veri senkronizasyonu (CDC) derdini bitirir. 2) Pinecone: Tamamen yönetilen (Serverless) SaaS'tır. Altyapı yönetmek istemeyen bulut ekipleri için idealdir ancak veriler şirket dışına çıkar. 3) Qdrant: Rust ile yazılmıştır, bellek verimliliği mükemmeldir ve karmaşık metadata filtrelemelerinde (Payload Indexing) rakipsizdir. 4) Milvus: 100 milyondan fazla vektörü Kubernetes üzerinde sharding yaparak dağıtık yönetmek için tasarlanmıştır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architectural Trade-Off Matrix: 1) pgvector: Unbeatable for systems <5M vectors where transactional ACID guarantees and relational SQL JOINs with user/permission tables dominate. Eliminates dual-write consistency issues. 2) Pinecone: Fully managed Serverless vector SaaS; minimizes operational toil but introduces third-party data sovereignty and egress concerns. 3) Qdrant: Bare-metal Rust engine delivering peerless memory efficiency and hardware-accelerated payload filtering. 4) Milvus: Distributed microservice architecture built for billion-scale embeddings across Kubernetes clusters.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Vektör Veritabanı Seçimini Belirleyen Karar Motoru
public class VectorStoreDecisionEngine {
    public enum VectorStoreChoice { PGVECTOR, PINECONE_SAAS, QDRANT_RUST, MILVUS_DISTRIBUTED }

    public static VectorStoreChoice evaluate(long totalVectors, boolean requiresRelationalJoin,
boolean isCloudSaaSPreferred) {
        if (totalVectors < 5_000_000 && requiresRelationalJoin) {
            return VectorStoreChoice.PGVECTOR; // Postgres ile devam et, mimariyi sade tut!
        }
        if (isCloudSaaSPreferred && totalVectors < 50_000_000) {
            return VectorStoreChoice.PINECONE_SAAS;
        }
        if (totalVectors > 100_000_000) {
            return VectorStoreChoice.MILVUS_DISTRIBUTED; // Milyar ölçekte dağıtık mimari
        }
        return VectorStoreChoice.QDRANT_RUST;
    }
}

void main() {
    var choice = VectorStoreDecisionEngine.evaluate(250_000, true, false);
    IO.println("Önerilen Vektör Veritabanı: " + choice); // PGVECTOR
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mülakat Uyarısı: Ayrı bir vektör veritabanı (ör. Pinecone) kullandığınızda, ana veritabanınızdaki (MySQL/Postgres) veriler güncellendiğinde veya silindiğinde iki veritabanı arasında tutarsızlık (Data Drift) oluşur. Bu senkronizasyonu sağlamak için Debezium CDC veya Outbox Pattern anlatmanız beklenir!

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Dual-Write consistency trap: Introducing a standalone vector store beside your relational database creates dual-write split-brain risks. Deleting a record in Postgres without an event-driven sync (CDC / Kafka) leaves orphaned embeddings in Pinecone.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• 5 Milyon vektörün altındaki çoğu senaryoda PostgreSQL + pgvector en pratik ve güçlü çözümdür.

• İlişkisel verilerle (Kullanıcı, İzinler, Roller) vektörleri aynı tabloda JOIN edebilmek büyük nimettir.

• Yüz milyonlarca vektör için Milvus veya Qdrant gibi özelleşmiş motorlara geçilmelidir.

🎯 Key Takeaways for the Interview

• For datasets under 5M vectors, PostgreSQL + pgvector eliminates dual-write architectural overhead.

• Co-locating relational permissions and vector embeddings prevents security filter bypasses.

• Billions of embeddings justify dedicated distributed vector engines like Milvus or Qdrant.

17

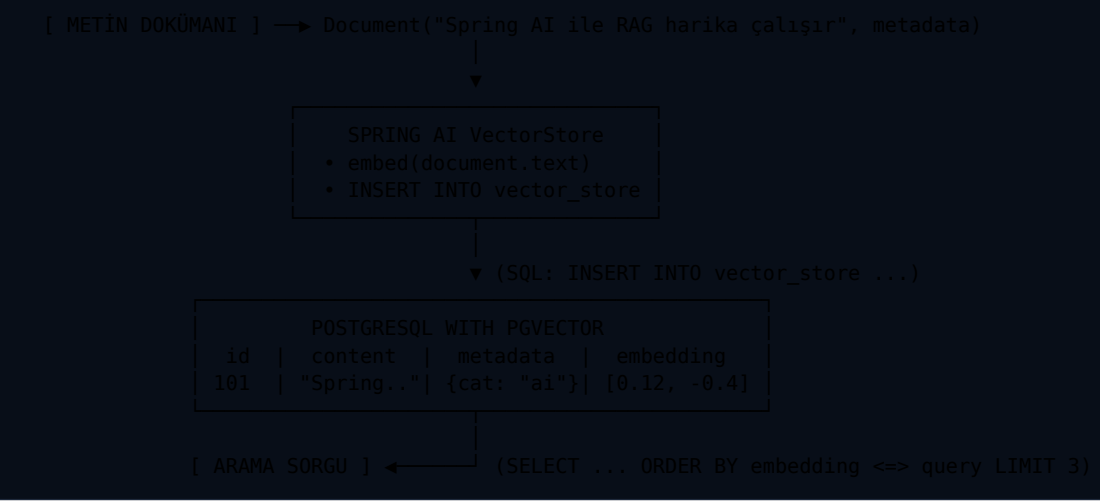
BÖLÜM: VEKTÖRLER, EMBEDDING'LER VE VEKTÖR VERITABANLARI | MODULE: VECTORS, EMBEDDINGS & VECTOR DATABASE ARCHITECTURE

Spring AI VectorStore: PostgreSQL pgvector Entegrasyonu ve Benzerlik Araması

Spring AI VectorStore: PostgreSQL pgvector Integration & Similarity Search

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Elinizde binlerce kartvizit olduğunu düşünün. Normalde bir kartviziti bulmak için adına bakarsınız. Spring AI'ın `VectorStore`'u ise kartvizitlerin arkasına görünmez bir manyetik çip yapıştırır. Siz 'Bana yapay zekadan anlayan bir avukat bul' dediğinizde, elinize bir mıknatıs alır ve kutunun üstünde gezdirirsiniz; aradığınız konsepte en yakın 3 kartvizit 'ŞLAAAP' diye mıknatısa yapışır! Spring AI tüm bu mıknatıs (vektör) işlerini sizin yerinize tek satır kodla yönetir!

🇬🇧 ENGLISH (ELI5)

Imagine a shoebox filled with 5,000 business cards. Spring AI's `VectorStore` attaches a semantic magnet to every card. When you query: 'Find me an attorney who specializes in international cloud compliance', you don't grep for exact keywords. You wave a query magnet over the box, and the 3 cards with the closest semantic magnetic polarity instantly snap onto your hand! Spring AI orchestrates the embedding, SQL insertion, and similarity queries behind a single unified interface!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Spring AI pgvector Entegrasyon Mekanizması: 1) Tablo Yapısı: `vector_store` tablosunda `id (UUID)`, `content (TEXT)`, `metadata (JSONB)` ve `embedding (vector(1536))` sütunları yer alır. 2) SearchRequest: `SearchRequest.query(...).withTopK(5).withSimilarityThreshold(0.75)` zinciri ile benzerlik eşiği ve sonuç sayısı ayarlanır. 3) Metadata Filtering: `withFilterExpression("category == 'finance' && year >= 2024")` ifadesi, SQL WHERE koşuluna dönüştürülerek vektör aramasından önce veya sonra (Pre/Post-filtering) işletilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Spring AI pgvector Architecture: 1) Schema Definition: Operates against standard tables hosting `id (UUID)`, `content (text)`, `metadata (jsonb)`, and `embedding (vector(1536))`. 2) SearchRequest: Fluent abstraction configuring `query()`, `topK()`, and `similarityThreshold()` (filtering out vectors with cosine similarity below a cutoff). 3) Native SQL Dialect Generation: Translates filter expressions into parameterized SQL WHERE clauses (`metadata->>'category' = 'finance'`), avoiding costly post-query in-memory filtering.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: pgvector VectorStore ile Benzerlik Araması
import org.springframework.ai.document.Document;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.ai.vectorstore.SearchRequest;
import java.util.List;
import java.util.Map;

public class KnowledgeBaseService {
    private final VectorStore vectorStore;

    public KnowledgeBaseService(VectorStore vectorStore) {
        this.vectorStore = vectorStore;
    }

    public void ingestDocument(String docId, String text, String department) {
        var document = new Document(docId, text, Map.of("department", department));
        vectorStore.add(List.of(document)); // Otomatik embed edilir ve pgvector'e yazılır!
    }

    public List<Document> searchRelevantKnowledge(String query, String department) {
        return vectorStore.similaritySearch(
            SearchRequest.builder()
                .query(query)
                .topK(3)
                .similarityThreshold(0.70)
                .filterExpression("department == '" + department + "'")
                .build()
        );
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Performans Darboğazı: `similarityThreshold` belirtmezseniz, aradığınız konuyla uzaktan yakından ALAKASI OLMAYAN dokümanlar bile ilk 3 sırada gelerek modelin bağlamını kirletir ve halüsinasyona yol açar! Daima makul bir eşik (ör. 0.70+) belirleyin.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Retrieval pitfall: Querying without a `similarityThreshold` forces the vector store to return the top K nearest neighbors even if their similarity score is abysmal (e.g. 0.20), poisoning the prompt context with irrelevant garbage.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Spring AI VectorStore, doküman yükleme ve vektör aramasını tekilleştiren bir SPI'dır.
- Metadata filtreleme, alakasız dokümanları SQL seviyesinde eleyerek arama doğruluğunu artırır.
- Benzerlik eşiği (Similarity Threshold) halüsinasyonu önlemede ilk savunma hattıdır.

🎯 Key Takeaways for the Interview

- Spring AI VectorStore unifies document ingestion, embedding, and ANN retrieval behind a clean SPI.
- Metadata filtering at the database layer drastically improves search relevance and speed.
- A well-calibrated similarity threshold is your first line of defense against hallucinations.

18

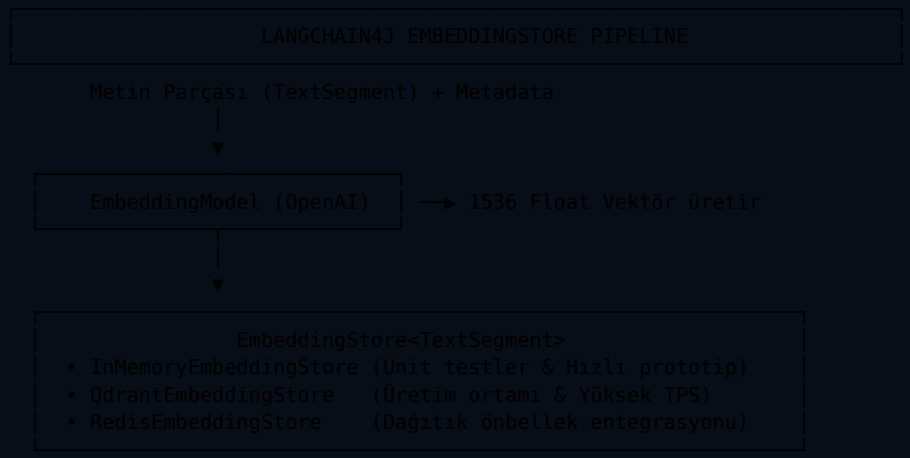
BÖLÜM: VEKTÖRLER, EMBEDDING'LER VE VEKTÖR VERITABANLARI | MODULE: VECTORS, EMBEDDINGS & VECTOR DATABASE ARCHITECTURE

LangChain4j EmbeddingStore: Bellek-İçi ve Dağıtık Vektör Yönetimi

LangChain4j EmbeddingStore: In-Memory, Qdrant & Redis Distributed Vector Management

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Yeni bir yapay zeka özelliği kodlarken her seferinde Docker ile koca bir PostgreSQL veritabanı ayağa kaldırmakla uğraşmak istemezsiniz (Tıpkı birim test yazarken H2 in-memory veritabanı kullanmak gibi). LangChain4j'in `InMemoryEmbeddingStore` yapısı sayesinde tek satır kodla RAM üzerinde çalışan geçici bir vektör veritabanı oluşturabilirsiniz! Testleriniz 1 saniyede biter. Canlıya (Production) çıkarken ise tek bir satırı değiştirip onu devasa bir Qdrant veya pgvector kümesine bağlarsınız!

🇬🇧 ENGLISH (ELI5)

When prototyping a new feature or running unit tests, spinning up external PostgreSQL clusters or Docker containers creates friction. LangChain4j provides `InMemoryEmbeddingStore`: an ephemeral vector store running directly inside JVM heap memory with zero dependencies! Your CI/CD unit tests execute in milliseconds. When shipping to production, you swap a single bean configuration to point to Qdrant or pgvector without touching your application logic!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

LangChain4j Vektör Mimarisi: 1) `TextSegment`: Vektörleştirilen metin parçasıdır; `Metadata` haritası ile zenginleştirilir. 2) `Embedding`: Ham `float[]` dizisini sarmalar. 3) `EmbeddingMatch\<TextSegment>`: Arama sonucunda dönen nesnedir; `score` (benzerlik puanı), `embeddingId` ve `embedded` (orijinal metin) taşır. 4) Serialization: LangChain4j bellek içi vektörleri JSON formatında diske kaydedip tekrar yükleme (`serializeToJsonFile` / `initFromFile`) desteğine sahiptir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

LangChain4j Vector Abstraction Internals: 1) `TextSegment`: Immutable domain object pairing text slices with contextual metadata maps. 2) `Embedding`: Strongly-typed wrapper encapsulating floating-point arrays. 3) `EmbeddingMatch\<TextSegment>`: Result envelope providing similarity scores, vector IDs, and reconstructed segments. 4) Portability & Persistence: The in-memory store can snapshot its graph and vectors directly to local disk via JSON serialization, enabling offline test fixtures.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & LangChain4j: In-Memory Vektör Deposu ve Benzerlik Araması
import dev.langchain4j.data.embedding.Embedding;
import dev.langchain4j.data.segment.TextSegment;
import dev.langchain4j.store.embedding.inmemory.InMemoryEmbeddingStore;
import dev.langchain4j.store.embedding.EmbeddingMatch;
import java.util.List;

void main() {
    var store = new InMemoryEmbeddingStore<TextSegment>();

    // Örnek segment ve vektör ekleme
    var segment1 = TextSegment.from("Kafka, dağıtık bir commit log mimarisine sahiptir.");
    var embedding1 = Embedding.from(new float[]{0.9f, 0.1f, 0.0f});
    store.add(embedding1, segment1);

    var segment2 = TextSegment.from("PostgreSQL, ilişkisel ve ACID uyumlu bir veritabanıdır.");
    var embedding2 = Embedding.from(new float[]{0.0f, 0.9f, 0.1f});
    store.add(embedding2, segment2);

    // Benzerlik araması: Kafka'ya yakın sorgu
    var queryEmbedding = Embedding.from(new float[]{0.85f, 0.15f, 0.0f});
    List<EmbeddingMatch<TextSegment>> matches = store.findRelevant(queryEmbedding, 1, 0.70);

    for (var match : matches) {
        IO.println("En Yakın Eşleşme (Skor: " + match.score() + "): " + match.embedded().text());
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Hafıza Uyarısı: `InMemoryEmbeddingStore` milyonlarca vektör için uygun DEĞİLDİR! Her 1536 boyutlu float vektör bellekte yaklaşık 6KB yer kaplar. 1 milyon vektör 6 GB saf RAM tüketir ve JVM Garbage Collector'ı kitleyebilir (GC Pauses).

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Memory threshold: Do not run `InMemoryEmbeddingStore` in production with >100,000 vectors. Storing dense 1,536-dimensional float arrays in heap memory causes massive JVM footprint and triggers fatal garbage collection pauses.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• InMemoryEmbeddingStore, birim testler ve hızlı prototipleme için benzersizdir.

• LangChain4j'in tip güvenli yapısı (TextSegment, EmbeddingMatch) temiz kod sağlar.

• Büyük ölçekli üretim ortamlarında mutlaka disk ve bellek optimizasyonlu harici motorlar kullanılmalıdır.

🎯 Key Takeaways for the Interview

• InMemoryEmbeddingStore is unbeatable for rapid prototyping and hermetic unit testing.

• Type-safe wrappers (TextSegment, EmbeddingMatch) ensure clean architectural decoupling.

• Production systems mandate external, specialized vector engines with persistence.